

Document on Sept 2, 2025

MEG/EEG Preprocessing Pipeline Documentation

Version: 1.3 (pipeline) • **Checkpoint format:** 1.0

Author: Gregory McCarthy with AI assistance

Last updated: 2025-09-02

Table of Contents

1. [Introduction](#)
 2. [Core Design Principles](#)
 3. [Stage 1: Automated Processing](#)
 4. [Stage 2: Interactive Finalization](#)
 5. [Execution Modes](#)
 6. [Quality Control Framework](#)
 7. [Configuration Guide](#)
 8. [Practical Usage](#)
 9. [Troubleshooting](#)
 10. [Appendix: Complete Configuration](#)
-

Introduction

Overview

This MEG/EEG preprocessing pipeline represents a sophisticated approach to neurophysiological data processing, balancing computational efficiency with scientific rigor. The system divides preprocessing into two distinct stages - automated heavy processing and interactive finalization - connected by a robust checkpoint system that ensures both reproducibility and flexibility.

The Challenge

Preprocessing MEG and EEG data presents unique challenges:

- **Computational intensity:** Operations like Maxwell filtering require significant resources
- **Human judgment requirements:** Artifact identification often needs expert oversight
- **Reproducibility demands:** Scientific rigor requires consistent, documented processing
- **Scale considerations:** Modern studies require processing hundreds or thousands of sessions
- **Flexibility needs:** Different analyses require different preprocessing choices

The Solution

This pipeline addresses these challenges through:

- **Two-stage architecture:** Separating automated from interactive processing
 - **Checkpoint system:** Enabling reproducible, resumable workflows
 - **Multiple execution modes:** Supporting local, HPC, and hybrid processing
 - **Comprehensive quality control:** Tracking data quality throughout processing
 - **Conservative philosophy:** Preserving raw signal characteristics until human review
-

Core Design Principles

Stage-Based Processing Philosophy

The pipeline implements a fundamental separation between computationally intensive automated processing and human-guided interactive refinement. This division reflects the reality that while many preprocessing steps can be standardized and automated, critical decisions about artifact rejection and component selection benefit from expert oversight.

Stage 1: Automated Heavy Processing

Stage 1 handles all deterministic, computationally intensive operations that don't require human judgment:

- **Head position estimation** from continuous HPI signals
- **Maxwell filtering** (SSS/tSSS) for environmental noise suppression
- **Line noise removal** via notch filtering
- **Automated artifact detection** using cross-validated thresholds
- **Quality metrics computation** for objective assessment
- **Checkpoint creation** for reproducibility and resumability

This stage is designed for unattended execution on HPC clusters or overnight local runs, producing a standardized baseline dataset ready for expert review.

Stage 2: Interactive Finalization

Stage 2 incorporates human expertise for nuanced decisions:

- **Validation** of automated artifact detection
- **Interactive review** of bad channels and epochs
- **ICA component selection** based on physiological understanding
- **Event extraction** with experiment-specific logic
- **Final filtering decisions** based on analysis requirements
- **Quality assurance** before final output

This stage requires display capabilities and user interaction, making it suitable for local workstations or interactive HPC sessions.

The Checkpoint System

Architecture and Purpose

The checkpoint system serves as the critical bridge between stages, ensuring that heavy computations aren't repeated while maintaining flexibility for different analysis approaches. Each checkpoint consists of two components:

Checkpoint FIF File (`*_desc-parproc_meg.fif`):

- Contains the partially processed data after Stage 1
- Includes all Maxwell filtering and notch filtering results
- Preserves full bandwidth for downstream flexibility
- Memory-mapped for efficient loading

Manifest YAML File (`*_desc-parproc_meg_manifest.yaml`):

- Documents all processing parameters and decisions
- Records configuration hash for reproducibility
- Lists all detected artifacts and bad channels
- References calibration files and diagnostic outputs

Benefits

The checkpoint system enables several critical capabilities:

- **Reproducibility**: Exact parameters and decisions are documented
- **Efficiency**: Heavy computations aren't repeated unnecessarily
- **Flexibility**: Stage 2 can be rerun with different parameters

- **Collaboration:** Checkpoints can be shared between researchers
- **Audit trails:** Complete processing history is preserved

Hybrid Processing Mode

Motivation

Research environments often feature powerful HPC clusters for computation alongside responsive local workstations for interactive work. Hybrid mode bridges these environments, enabling researchers to leverage HPC resources while maintaining interactive responsiveness.

Implementation

Hybrid mode orchestrates a sophisticated workflow:

1. **Prefetch Phase:** Raw data and existing derivatives are synchronized from HPC to local cache
2. **Local Processing:** Both stages run on the local workstation with full interactivity
3. **Pushback Phase:** Completed derivatives are synchronized back to HPC storage

This approach ensures:

- HPC remains the authoritative data source
- Local processing provides optimal interactivity
- Network overhead is minimized through intelligent caching
- Multiple researchers can work independently

Filtering Philosophy

Strategic Application

The pipeline implements a sophisticated filtering strategy that balances signal preservation with algorithm requirements:

- **No redundant filtering:** Data is never filtered twice unless explicitly requested
- **Algorithm-specific copies:** Filtered copies are created for specific algorithms
- **Original preservation:** Full-band data is maintained for final outputs
- **Explicit documentation:** All filtering operations are logged

Filtering Strategy Table

Processing Step	Filter Applied	Data Preserved	Rationale
AutoReject computation	1-40 Hz bandpass	No (temporary copy)	Stabilizes peak-to-peak calculations
ICA fitting	1-40 Hz + resample	No (temporary copy)	Reduces computational load and noise
ICA application	None (original data)	Yes	Preserves full frequency content
Final output	User-specified	Yes	Tailored to analysis needs

Stage 1: Automated Processing

Raw Data Loading

Overview

The pipeline begins by loading MEG/EEG data from a BIDS-compliant directory structure. This standardized format ensures consistency across datasets and facilitates data sharing.

Implementation Details

The loading process handles several complexities:

- **Split file support:** Automatically detects and concatenates multi-part recordings
- **Memory mapping:** Efficient I/O without loading entire datasets into RAM
- **BIDS validation:** Ensures compliance with BIDS specifications
- **Metadata extraction:** Captures all experimental parameters

Configuration

```
subject: "001"
session: "01"
run: "02"
task: "fairy"
bids_root: /path/to/bids/dataset
```

Head Position Estimation and Movement Compensation

The Problem

Subject head movement during MEG recordings can significantly degrade data quality by:

- Changing the relationship between sensors and sources
- Introducing movement-correlated artifacts
- Reducing spatial resolution
- Complicating source localization

The Solution

The pipeline implements comprehensive movement tracking and compensation:

Continuous HPI Monitoring:

- Extracts positions from HPI coils throughout recording
- Generates time-resolved position traces
- Computes movement statistics and quality metrics
- Creates diagnostic visualizations

Movement Compensation via Maxwell Filtering:

- Applies physics-based field separation
- Compensates for head movement using position traces
- Maintains optimal sensor-source relationships
- Preserves signal integrity during compensation

Quality Assessment

Movement tracking produces several diagnostic outputs:

- Time-series plots of translation and rotation
- Movement velocity histograms
- Quality scores for HPI coil fits
- Statistical summaries of movement parameters

Maxwell Filtering (SSS/tSSS)

Theoretical Foundation

Maxwell filtering leverages fundamental electromagnetic theory to separate brain signals from environmental noise. The method decomposes measured fields into:

- **Internal components:** Originating from within the sensor array (brain signals)
- **External components:** Originating from outside the array (environmental noise)

This physics-based separation provides superior noise suppression compared to purely statistical methods.

Implementation

The pipeline implements both SSS (Signal Space Separation) and tSSS (temporal extension):

SSS Processing:

- Spherical harmonic decomposition of magnetic fields
- Separation of internal/external field components
- Reconstruction using only internal components
- Bad channel interpolation via field reconstruction

tSSS Enhancement:

- Temporal correlation analysis across channels
- Identification of spatially-correlated noise patterns
- Suppression of movement-related artifacts
- Preservation of neural signal dynamics

Configuration Requirements

Maxwell filtering requires system-specific calibration:

```
calibration_file: cal/sss_cal_factory_20230619.dat
cross_talk_file: cal/ct_sparse_triux2.fif
head_movement:
  enabled: true
head_position_origin: null # or specify reference position
```

EEG Channel Configuration

Challenges in EEG Processing

EEG data within MEG systems presents unique challenges:

- Non-standard electrode positions
- Variable montage configurations
- Inconsistent channel naming conventions
- Missing or incorrect sensor locations

Standardization Process

The pipeline implements comprehensive EEG standardization:

1. **Montage Application:** Loads electrode positions from standard files
2. **Channel Validation:** Verifies all channels have valid locations
3. **Label Standardization:** Converts to consistent naming conventions
4. **Bad Channel Detection:** Identifies electrodes with poor contact
5. **Reference Verification:** Ensures proper reference configuration

Metadata Repair System

Motivation

Real-world MEG/EEG data often contains metadata inconsistencies that can disrupt processing or analysis. The pipeline implements a flexible repair system to address these issues programmatically.

Repair Categories

Systematic Fixes:

- Corrects mislabeled channel types
- Standardizes acquisition parameters
- Fills missing metadata fields
- Aligns coordinate systems

Expert Patches:

- Custom Python code for dataset-specific issues
- Executed within the data context
- Full access to raw data structures
- Logged for reproducibility

Configuration

```
metadata_fixes:
  fix_non_eeg_channels: {}
  fix_generic: {}
  expert_patch: |
    # Custom Python code here
    print("✅ Expert patch executed")
```

Line Noise Removal

The Challenge

Power line interference (50/60 Hz and harmonics) represents a pervasive noise source that can obscure neural signals. While Maxwell filtering removes much environmental noise, targeted line noise suppression often remains necessary.

Notch Filtering Strategy

The pipeline implements intelligent notch filtering:

- **Fundamental and harmonics:** Targets line frequency and integer multiples
- **Adaptive width:** Adjusts filter width based on spectral peaks
- **Quality preservation:** Minimizes impact on adjacent frequencies
- **Verification:** Generates before/after spectral plots

AutoReject: Automated Artifact Detection

Overview and Theoretical Foundation

AutoReject represents a paradigm shift in artifact detection, moving from subjective manual annotation to objective, data-driven thresholds. Developed by Jas et al. (2017), the algorithm addresses fundamental limitations of traditional approaches.

The Problem AutoReject Solves

Traditional artifact detection suffers from:

- **Subjective variability:** Different experts apply different criteria
- **Time intensity:** Manual review doesn't scale to large datasets
- **Threshold uncertainty:** Optimal peak-to-peak values vary across systems
- **Inconsistent criteria:** Standards differ between laboratories

How AutoReject Works

The algorithm employs cross-validation to learn optimal detection thresholds from the data itself:

Global Mode:

- Estimates single threshold for all sensors
- Uses cross-validation comparing training/validation sets
- Robust to outliers via median-based metrics
- Suitable for catastrophic artifact detection

Local Mode:

- Learns sensor-specific thresholds
- Accounts for amplitude differences across channels
- Creates detailed artifact maps
- Balances interpolation vs. rejection

Data Augmentation:

- Handles globally bad sensors via virtual interpolation
- Leave-one-out augmentation doubles effective dataset size
- Ensures robust validation even with compromised sensors

Three-Layer Detection Architecture

The pipeline implements a hierarchical detection strategy:

Layer 1: RANSAC Bad Channel Detection

- **Purpose:** Identify globally compromised channels
- **Method:** Consensus-based outlier detection via random sampling
- **Targets:** Systematic failures, environmental contamination
- **Approach:** Conservative detection of obvious failures

Layer 2: AutoReject Epoch Analysis

- **Purpose:** Detect intermittent and sensor-specific artifacts
- **Method:** Cross-validated threshold learning
- **Targets:** Eye blinks, muscle activity, transient noise
- **Approach:** Adaptive thresholds based on data characteristics

Layer 3: Global Epoch Consensus

- **Purpose:** Identify trials with excessive artifact burden
- **Method:** Voting across sensors with configurable thresholds
- **Targets:** Widespread contamination events
- **Approach:** Safety net for heavily corrupted trials

Detection-Only Philosophy

This implementation uses AutoReject purely for annotation, preserving several advantages:

- **Signal preservation:** No automatic interpolation or data modification

- **Human oversight:** Expert review before any reconstruction
- **Conservative bias:** Prevents inadvertent signal alteration
- **Flexibility:** Reconstruction methods can be chosen per analysis

Configuration

```
autoreject:
  enabled: true
  which_types: [eeg, mag]      # Channel types to analyze

  filter:
    highpass: 1.0              # Remove slow drifts
    lowpass: 40.0              # Remove high-frequency noise
    resample_hz: null          # Optional downsampling

  epoch:
    duration: 2.0              # Epoch length in seconds
    tmin: 0.0                  # Start time
    tmax: 2.0                  # End time

  cv_folds: 5                  # Cross-validation folds
  thresh_method: 'random_search' # Or 'bayesian_optimization'
  n_interpolate: [0]           # [0] = detection only

  consensus_thresh: 0.3        # 30% agreement for bad epochs
  global_epoch_thresh: 0.5     # 50% bad sensors → reject epoch
```

Quality Control Metrics

Comprehensive Assessment Framework

Quality control runs at multiple pipeline stages, providing objective metrics for data quality assessment. The system computes metrics both before and after Maxwell filtering, enabling quantitative evaluation of preprocessing effectiveness.

Metric Categories

Amplitude Statistics (per channel type):

- RMS (Root Mean Square) values
- 95th percentile peak-to-peak amplitudes
- Variance measurements
- All computed on demeaned data

Spectral Metrics:

- High-frequency noise (200-250 Hz band)
- Line noise peaks and harmonic ratios
- Neural band power (delta through gamma)
- $1/f$ slope and intercept (with alpha notched)

Signal Quality Indicators:

- Kurtosis for non-Gaussianity detection
- cHPI coil status and quality
- Recording metadata integrity
- SSS reconstruction quality

Improvement Metrics (pre/post Maxwell):

- Shielding factor (linear, dB, percentage)
- Signal preservation (alpha/beta bands)
- Amplitude and variance ratios
- Kurtosis reduction
- Overall quality grade

Units and Reporting

- **Storage:** SI units (Tesla, Tesla/meter, Volts)
- **Display:** Intuitive units (fT, fT/cm, μ V)
- **Logging:** Both formats for clarity

Checkpoint Creation

Final Stage 1 Output

Stage 1 concludes by writing comprehensive checkpoint files:

Checkpoint FIF (`*_desc-parproc_meg.fif`):

- Partially processed data after Maxwell and notch filtering
- All detected artifacts annotated but not removed
- Full bandwidth preserved for Stage 2 flexibility
- Memory-mapped for efficient loading

Manifest YAML (`*_desc-parproc_meg_manifest.yaml`):

- Complete processing parameters
 - Configuration hash for reproducibility
 - Lists of bad channels and epochs
 - References to calibration files
 - Paths to diagnostic figures
 - Quality control metrics
-

Stage 2: Interactive Finalization

Checkpoint Validation and Loading

Verification Process

Stage 2 begins with comprehensive checkpoint validation:

- **File existence:** Confirms both FIF and manifest are present
- **Integrity checks:** Validates file formats and readability
- **Version compatibility:** Ensures checkpoint format matches pipeline
- **Configuration comparison:** Detects parameter changes since Stage 1

Loading Strategy

The validated checkpoint becomes the starting point for Stage 2 processing, with all Stage 1 decisions and annotations preserved but modifiable.

Interactive Bad Channel and Epoch Review

The Human Expert Advantage

While automated detection excels at consistency and efficiency, human experts bring:

- Physiological understanding of artifact sources
- Context awareness for experimental paradigms
- Pattern recognition across sessions
- Judgment for edge cases

Interactive Review Process

The pipeline provides sophisticated visualization tools:

Channel Review Interface:

- Time-series visualization with adjustable scales
- Topographic maps showing spatial patterns
- Spectral plots for frequency-domain assessment
- Statistical outlier highlighting

Epoch Review Interface:

- Butterfly plots for multi-channel overview
- Individual channel traces
- Event markers and annotations
- Keyboard shortcuts for efficient review

Decision Documentation

All manual modifications are logged:

- Original automated decisions
- User modifications with timestamps
- Justification notes (optional)
- Reviewer identification

Independent Component Analysis (ICA)

Theoretical Foundation

ICA decomposes multichannel data into statistically independent components, effectively separating:

- Neural sources with distinct spatial patterns
- Physiological artifacts (cardiac, ocular, muscular)
- Environmental noise sources
- Sensor-specific artifacts

Implementation Strategy

The pipeline implements a sophisticated ICA workflow:

Preprocessing for Stability:

1. Create filtered copy (1-40 Hz) for cleaner decomposition
2. Resample to reduce computational load
3. Apply time limit for long recordings
4. Use consistent random seeds for reproducibility

Component Fitting:

- Separate ICA for EEG and MEG modalities
- Automatic component number selection
- Convergence monitoring
- Quality metrics for decomposition

Application to Original Data:

- ICA weights derived from filtered data
- Applied to full-bandwidth original
- Preserves all frequency content
- No information loss

Interactive Component Selection

Component review interfaces provide:

Visualization Tools:

- Component topographies
- Time courses
- Frequency spectra
- Event-locked averages

Classification Aids:

- Correlation with EOG/ECG channels
- Spectral fingerprints
- Spatial patterns
- Automated suggestions (not decisions)

Configuration

```
ica_preprocessing:
  eeg:
    highpass: 1.0
    lowpass: 40.0
    resample_hz: 200
    max_ica_duration_sec: 1800
    interactive: true
    random_state: 97
  meg:
```

```
highpass: 1.0
lowpass: 40.0
resample_hz: 200
max_ica_duration_sec: 1800
interactive: true
random_state: 97
```

Event Detection and Extraction

Flexible Event Handling

Experiments use diverse event coding schemes. The pipeline provides flexible extraction:

Trigger Channel Processing:

- Configurable bitmask for relevant bits
- Merge with annotation events
- Temporal alignment verification
- Count validation

Event Validation:

- Expected event sequences
- Timing consistency checks
- Missing event detection
- Duplicate removal

Configuration

```
stim_include_mask: 0xFFFF # Use all 16 bits
# Or more selective: 0xC000 for bits 14-15 only
```

Final Filtering and Channel Management

Analysis-Specific Preparation

Different analyses require different frequency bands and channel selections:

Filtering Options:

- High-pass for slow drift removal
- Low-pass for high-frequency noise
- Band-pass for specific frequency analysis

- Notch for residual line noise

Channel Management:

- Remove auxiliary channels (HPI, etc.)
- Select analysis-specific channel sets
- Ensure consistent channel ordering
- Document all exclusions

Intelligent Channel Dropping

The pipeline uses prefix matching for auxiliary channels:

```
final_filter:
  drop_channel_types:
    - hpi      # HPI coils
    - hlc      # Head localization coils
    - coil     # Other coil types
```

Output Generation

Comprehensive Deliverables

Stage 2 produces complete preprocessing outputs:

Preprocessed Data (*_desc-preproc_meg.fif):

- Fully cleaned and filtered data
- All artifacts removed or annotated
- Ready for analysis

ICA Solutions (*_desc-preprocICA{eeg,meg}.fif):

- Complete decomposition matrices
- Component metadata
- Reusable for future analysis

Documentation:

- Complete processing logs (YAML and JSON)
- Quality control reports
- Diagnostic figures
- Processing timeline

Hybrid Mode Synchronization

Pushback to HPC

In hybrid mode, Stage 2 concludes by synchronizing results:

1. **Validation:** Ensures all outputs are complete
 2. **Packaging:** Organizes files for transfer
 3. **Transfer:** Secure copy to HPC storage
 4. **Verification:** Confirms successful transfer
 5. **Cleanup:** Optional local cache management
-

Execution Modes

Local Processing

Use Case

Local processing suits scenarios with:

- Moderate data sizes
- Full pipeline execution needs
- Interactive requirements throughout
- Development and testing

Workflow

```
python pipeline.py config/local.yaml
```

The pipeline automatically:

1. Detects checkpoint presence
2. Runs appropriate stage(s)
3. Provides interactive interfaces
4. Saves all outputs locally

HPC Batch Processing (SLURM)

Use Case

SLURM batch processing excels for:

- Large-scale studies
- Overnight Stage 1 processing
- Parallel subject processing
- Resource-intensive computations

Implementation

```
#!/bin/bash
#SBATCH --job-name=meg_stage1
#SBATCH --output=logs/meg_%j.out
#SBATCH --time=04:00:00
#SBATCH --mem=64G
#SBATCH --cpus-per-task=8

export MPLBACKEND=Agg # Headless mode
python pipeline.py config/slurm.yaml --force-stage stage1
```

Key Considerations

- Stage 1 only (no interactivity)
- Headless matplotlib backend
- Checkpoint exit enabled
- Resource allocation planning

HPC Interactive (OOD)

Use Case

Open OnDemand sessions provide:

- GUI access on HPC resources
- Full pipeline capability
- Direct access to HPC storage
- Collaborative processing

Workflow

1. Request OOD desktop session
2. Navigate to pipeline directory
3. Run with display-enabled config

4. Complete both stages interactively

Hybrid Mode

Architecture

Hybrid mode orchestrates sophisticated data movement:

```
[HPC Storage] → [Prefetch] → [Local Cache]
                        ↓
                    [Local Processing]
                        ↓
[HPC Storage] ← [Pushback] ← [Local Results]
```

Configuration

```
remote_io:
  enabled: true
  hpc_host: transfer.cluster.edu
  hpc_user: username
  remote_bids_root: /path/on/hpc
  local_temp_dir: ./temp
```

Advantages

- Combines HPC storage with local interactivity
- Minimizes network latency during processing
- Maintains data integrity and versioning
- Enables distributed team workflows

Quality Control Framework

Multi-Stage Assessment

Quality control operates throughout the pipeline, providing objective metrics at critical junctures:

Pre-Processing Baseline

Establishes initial data quality:

- Raw amplitude distributions

- Baseline spectral characteristics
- Channel integrity assessment
- Recording metadata validation

Post-Maxwell Improvement

Quantifies noise suppression effectiveness:

- Shielding factors
- Signal preservation ratios
- Spectral cleaning metrics
- Spatial pattern improvements

Post-Artifact Removal

Validates cleaning effectiveness:

- Artifact reduction statistics
- Signal-to-noise improvements
- Component removal impacts
- Final quality scores

Metric Interpretation Guide

Amplitude Metrics

RMS (Root Mean Square):

- Expected ranges: MEG magnetometers (20-200 fT), MEG gradiometers (5-50 fT/cm), EEG (5-50 μ V)
- Outliers indicate: Bad channels, environmental noise, movement artifacts

Peak-to-Peak 95th Percentile:

- Robust artifact indicator
- Less sensitive to outliers than maximum
- Useful for threshold setting

Spectral Metrics

1/f Slope:

- Typical range: -0.5 to -2.0

- Flatter slopes suggest: White noise contamination
- Steeper slopes suggest: Low-frequency artifacts

Alpha Peak SNR:

- Healthy adults: >3 dB above background
- Indicates: Data quality and subject state
- Useful for: Vigilance assessment

Line Noise Ratios:

- Pre-notch: Often >10x background
- Post-notch: Should approach 1.0
- Residual peaks: Indicate filter limitations

Statistical Metrics

Kurtosis:

- Normal range: 2.5-3.5
- High values (>5): Spike artifacts
- Low values (<2): Unusual distributions

Variance Ratios (post/pre Maxwell):

- Good suppression: <0.1
- Moderate suppression: 0.1-0.3
- Poor suppression: >0.3

Quality Grades

The pipeline assigns overall quality grades:

- **A:** Excellent - minimal artifacts, good SNR
- **B:** Good - moderate artifacts, acceptable SNR
- **C:** Fair - significant artifacts, marginal SNR
- **D:** Poor - severe artifacts, low SNR
- **F:** Fail - unusable data

Configuration Guide

Essential Parameters

Subject Identification

```
subject: "001"  
session: "01"  
run: "02"  
task: "experiment"  
bids_root: /path/to/dataset
```

Processing Control

```
checkpoint:  
  enabled: true  
  exit_after_checkpoint: false # true for SLURM  
  
interactive_bad_channels: true  
interactive_ica: true
```

Advanced Configuration

Customized AutoReject

For stringent artifact detection:

```
autoreject:  
  cv_folds: 10 # More robust validation  
  thresh_method: 'bayesian_optimization'  
  consensus_thresh: 0.2 # More sensitive  
  global_epoch_thresh: 0.4 # Stricter rejection
```

For permissive detection:

```
autoreject:  
  cv_folds: 3 # Faster, less conservative  
  thresh_method: 'random_search'  
  consensus_thresh: 0.5 # Less sensitive  
  global_epoch_thresh: 0.7 # More permissive
```

ICA Optimization

For comprehensive decomposition:

```
ica_preprocessing:
  meg:
    max_ica_duration_sec: null      # No time limit
    random_state: 42                # Reproducibility
```

For rapid processing:

```
ica_preprocessing:
  meg:
    max_ica_duration_sec: 600      # 10-minute cap
    resample_hz: 100               # Aggressive downsampling
```

Configuration Profiles

Development Profile

Optimized for testing and debugging:

```
checkpoint:
  backup_before_overwrite: true
  validate_checkpoint_integrity: true

autoreject:
  cv_folds: 3
  thresh_method: 'random_search'

interactive_bad_channels: true
interactive_ica: true
```

Production Profile

Optimized for final analyses:

```
checkpoint:
  atomic_writes: true
  allow_resume_with_different_yaml: false

autoreject:
  cv_folds: 5
  thresh_method: 'bayesian_optimization'

repro:
```



```
random_state_global: 42
record_environment: true
```

HPC Batch Profile

Optimized for cluster execution:

```
checkpoint:
  exit_after_checkpoint: true

interactive_bad_channels: false
interactive_ica: false

remote_io:
  enabled: false
```

Practical Usage

Single Subject Processing

Local Execution

```
# Complete pipeline
python pipeline.py config/subject_001.yaml

# Stage 1 only
python pipeline.py config/subject_001.yaml --force-stage stage1

# Stage 2 from checkpoint
python pipeline.py config/subject_001.yaml --force-stage stage2
```

Monitoring Progress

The pipeline provides detailed progress information:

- Stage transitions
- Current processing step
- Time estimates
- Memory usage
- File I/O operations

Batch Processing

SLURM Array Jobs

For multiple subjects:

```
#!/bin/bash
#SBATCH --array=1-100
#SBATCH --job-name=meg_batch

SUBJECT=$(printf "%03d" $SLURM_ARRAY_TASK_ID)
sed "s/subject: .*/subject: \"${SUBJECT}\"/" \
    config/template.yaml > config/temp_${SUBJECT}.yaml

python pipeline.py config/temp_${SUBJECT}.yaml --force-stage stage1
```

Parallel Local Processing

Using GNU Parallel:

```
parallel -j 4 python pipeline.py config/subject_{}.yaml ::: 001 002 003 004
```

Quality Assurance Workflow

Systematic Review Process

1. Stage 1 Completion Review:

- Check diagnostic plots
- Verify Maxwell filtering effectiveness
- Review AutoReject statistics
- Assess overall data quality

2. Interactive Artifact Review:

- Examine flagged channels
- Validate epoch rejections
- Adjust thresholds if needed
- Document decisions

3. ICA Component Review:

- Identify artifact components
- Verify neural component preservation
- Cross-reference with behavior

- Apply conservative exclusion

4. **Final Validation:**

- Check event counts
- Verify filtering parameters
- Examine final PSDs
- Generate quality reports

Team Collaboration

Shared Processing

Teams can coordinate processing using hybrid mode:

1. **Central Configuration:**

```
remote_io:  
  remote_bids_root: /shared/project/bids
```

2. **Distributed Stage 1:**

- Team members claim subjects
- Run Stage 1 independently
- Push checkpoints to shared storage

3. **Expert Review:**

- Senior researchers review checkpoints
- Provide Stage 2 guidance
- Ensure consistency across team

4. **Final Processing:**

- Apply consistent Stage 2 parameters
- Generate standardized outputs
- Maintain processing logs

Troubleshooting

Common Issues and Solutions

Checkpoint Not Found

Symptoms: Pipeline attempts Stage 2 but can't find checkpoint

Diagnostics:

```
# Check checkpoint location
ls derivatives/preprocessing/sub-*/meg/*_desc-parproc_meg.fif

# Verify manifest
ls derivatives/preprocessing/sub-*/meg/*_manifest.yaml
```

Solutions:

- Verify derivatives path in config
- Ensure Stage 1 completed successfully
- Check file permissions
- Confirm subject/session/run identifiers match

Memory Errors

Symptoms: Out of memory during processing

Solutions:

```
# Reduce memory usage
autoreject:
  filter:
    resample_hz: 100 # Aggressive downsampling
  epoch:
    duration: 1.0 # Shorter epochs

ica_preprocessing:
  meg:
    max_ica_duration_sec: 900 # Limit data for ICA
```

GUI Display Issues

Symptoms: Interactive windows don't appear

Diagnostics:

```
# Check display
echo $DISPLAY

# Test matplotlib backend
python -c "import matplotlib; print(matplotlib.get_backend())"
```

Solutions:

- For local: Ensure X11/display server running
- For SSH: Use `ssh -X` or `ssh -Y`
- For SLURM: Set interactive mode only
- For OOD: Check VNC/desktop session

Calibration File Errors

Symptoms: Maxwell filtering fails with calibration errors

Solutions:

1. Use absolute paths for calibration files
2. Verify files exist on compute nodes
3. Check file permissions
4. Ensure correct system-specific files

Network Issues in Hybrid Mode

Symptoms: Prefetch/pushback operations fail or hang

Diagnostics:

```
# Test SSH connection
ssh user@hpc.cluster.edu "echo 'Connection successful'"

# Check rsync
rsync --version

# Test small transfer
rsync -av testfile user@hpc:/path/
```

Solutions:

- Configure SSH keys for passwordless access
- Increase timeout values for large files
- Use compression for slow connections
- Consider chunked transfers for reliability

AutoReject Failures

Symptoms: AutoReject produces unexpected results or fails

Common Causes and Solutions:

Too few epochs:

```
autoreject:
  epoch:
    duration: 1.0 # Shorter epochs = more samples
```

Inappropriate filtering:

```
autoreject:
  filter:
    highpass: 0.5 # Less aggressive for slow signals
    lowpass: 45.0 # Include more signal content
```

Wrong channel types:

```
autoreject:
  which_types: [mag, grad, eeg] # Include all present types
```

Performance Optimization

Computational Efficiency

CPU Optimization:

```
# Set thread counts
export OMP_NUM_THREADS=8
export MKL_NUM_THREADS=8
export OPENBLAS_NUM_THREADS=8
```

Memory Management:

- Use memory mapping for large files
- Process in chunks when possible
- Clear unnecessary variables
- Monitor usage with `htop` or `top`

I/O Optimization:

- Use local scratch for temporary files
- Minimize network filesystem access
- Batch small file operations

- Use SSD storage when available

Processing Speed Tips

Faster Stage 1:

- Reduce AutoReject CV folds
- Use random_search instead of Bayesian
- Limit epoch duration
- Downsample aggressively for AutoReject

Faster Stage 2:

- Limit ICA duration
- Pre-select bad channels
- Use component templates
- Batch similar subjects

Validation and Quality Checks

Data Integrity Verification

Checkpoint Validation:

```
# Quick validation script
import mne
import yaml

# Load checkpoint
raw = mne.io.read_raw_fif('checkpoint.fif')
with open('manifest.yaml') as f:
    manifest = yaml.safe_load(f)

# Verify consistency
assert len(raw.info['bads']) == len(manifest['artifacts']['bad_channels'])
print(f"✓ {len(raw.info['bads'])} bad channels")
```

Output Verification:

- Check file sizes are reasonable
- Verify all expected outputs exist
- Compare event counts before/after
- Validate sampling rates

Quality Metrics Interpretation

Warning Signs:

- Excessive bad channels (>10% of total)
- Very high kurtosis (>10)
- Poor Maxwell shielding (<50%)
- Missing alpha peaks in rest data
- Excessive epochs rejected (>30%)

Acceptable Variations:

- Higher noise in temporal channels
- Lower SNR in deep sources
- Session-to-session amplitude variations
- Subject-specific artifact patterns

Advanced Troubleshooting

Debug Mode

Enable verbose logging:

```
python pipeline.py config.yaml --verbose
```

Add debugging to config:

```
debug:  
  save_intermediate: true  
  plot_all_steps: true  
  verbose_autoreject: true
```

Pipeline State Recovery

If processing is interrupted:

1. Check partial outputs:

```
find derivatives/ -name "*.fif" -mmin -60
```

2. Verify checkpoint state:

```
# Check if checkpoint is valid  
try:
```



```

raw = mne.io.read_raw_fif('checkpoint.fif')
print("Checkpoint valid")
except:
    print("Checkpoint corrupted")

```

3. Recovery options:

- Resume from last checkpoint
- Restart Stage 1 if checkpoint invalid
- Use backup checkpoint if available

Custom Modifications

For special requirements, modify the pipeline:

Custom artifact detection:

```

# In expert_patch section
def custom_artifact_detection(raw):
    # Your custom logic here
    return bad_channels, bad_epochs

raw.info['bads'].extend(custom_artifact_detection(raw)[0])

```

Additional preprocessing steps:

```

metadata_fixes:
  expert_patch: |
    # Add custom filtering
    from mne.filter import filter_data
    raw._data = filter_data(raw._data, raw.info['sfreq'],
                            l_freq=0.3, h_freq=None)

```

Appendix: Complete Configuration

Full Production Configuration Example

```

# =====
# MEG/EEG Preprocessing Pipeline – Production Configuration
# Version: 1.3
# =====

```

```

# ----- SUBJECT IDENTIFICATION -----
subject: "001"
session: "01"
run: "02"
task: "experiment"

# ----- DATA PATHS -----
# Primary BIDS root (local or HPC)
bids_root: /data/project/bids

# ----- HYBRID MODE CONFIGURATION -----
# Enable for HPC storage with local processing
remote_io:
    enabled: false                # Set true for hybrid mode
    hpc_host: transfer.cluster.edu
    hpc_user: username
    remote_bids_root: /hpc/project/bids
    local_temp_dir: ./temp

# ----- HEAD MOVEMENT COMPENSATION -----
head_movement:
    enabled: true                # Use cHPI for movement tracking

# ----- MAXWELL FILTERING (SSS/tSSS) -----
# System-specific calibration files (required)
calibration_file: cals/sss_cal.dat
cross_talk_file: cals/ct_sparse.fif

# Origin for head position (null = use device coordinates)
head_position_origin: null

# ----- LINE NOISE CONFIGURATION -----
line_freq: 60.0                # 60 Hz for North America, 50 Hz
                                # for Europe

# ----- EEG CONFIGURATION -----
eeg_handling:
    montage: montage/standard_64ch.sfp    # Path to montage file

# ----- INTERACTIVITY SETTINGS -----
# Control which stages require user interaction
interactive_bad_channels: true        # Manual review of bad channels
interactive_ica: true                # Manual ICA component selection

# Optional manual overrides (usually empty)
manual_bad_channels: []

```

```

ica_exclude_eeg: []
ica_exclude_meg: []

# ----- AUTOREJECT CONFIGURATION -----
autoreject:
    # Global enable/disable
    enabled: true

    # Channel types to process
    which_types: [eeg, mag, grad]

    # Preprocessing for artifact detection
    filter:
        highpass: 1.0                # Hz - Remove slow drifts
        lowpass: 40.0                # Hz - Remove high-frequency noise
        resample_hz: null          # Optional downsampling

    # Epoching parameters
    epoch:
        duration: 2.0                # Seconds per epoch
        tmin: 0.0                    # Epoch start time
        tmax: 2.0                    # Epoch end time

    # Algorithm parameters
    cv_folds: 5                      # Cross-validation folds
    thresh_method: 'bayesian_optimization' # Or 'random_search' for speed
    n_interpolate: [0]              # [0] for detection only

    # Detection thresholds
    consensus_thresh: 0.3            # Fraction for bad epoch marking
    global_epoch_thresh: 0.5         # Fraction for epoch rejection

# ----- ICA CONFIGURATION -----
ica_preprocessing:
    # EEG-specific ICA parameters
    eeg:
        highpass: 1.0
        lowpass: 40.0
        resample_hz: 200
        max_ica_duration_sec: 1800    # 30-minute cap
        interactive: true
        random_state: 97

    # MEG-specific ICA parameters
    meg:
        highpass: 1.0

```

```

    lowpass: 40.0
    resample_hz: 200
    max_ica_duration_sec: 1800
    interactive: true
    random_state: 97

# ----- EVENT EXTRACTION -----
# Bitmask for trigger channel (0xFFFF = all 16 bits)
stim_include_mask: 0xFFFF

# ----- FINAL OUTPUT FILTERING -----
final_filter:
    highpass: null                # Apply high-pass if needed
    lowpass: null                 # Apply low-pass if needed
    resample_hz: null             # Final sampling rate
    drop_channel_types:           # Remove auxiliary channels
        - hpi
        - hlc
        - coil

# ----- METADATA REPAIR -----
metadata_fixes:
    fix_non_eeg_channels: {}      # Channel type corrections
    fix_generic: {}              # Generic fixes
    expert_patch: |               # Custom Python code
        # Add dataset-specific fixes here
    print("✅ Metadata repair completed")

# ----- CHECKPOINT CONFIGURATION -----
checkpoint:
    enabled: true                 # Always use checkpointing
    exit_after_checkpoint: false  # Set true for SLURM Stage 1
    derivatives_root: derivatives
    pipeline_name: preprocessing
    atomic_writes: true           # Prevent corruption
    allow_resume_with_different_yaml: true
    validate_checkpoint_integrity: true
    backup_before_overwrite: false
    max_checkpoint_age_hours: 168 # 1 week

# ----- REPRODUCIBILITY -----
repro:
    random_state_global: 42       # Global random seed
    record_environment: true       # Log software versions

```

Configuration Quick Reference

Parameter	Purpose	Typical Values
subject	Subject identifier	"001", "002", etc.
session	Session label	"01", "pre", "post"
run	Run number	"01", "02", etc.
task	Task name	"rest", "task", experiment-specific
bids_root	Dataset location	Absolute path
remote_io.enabled	Hybrid mode	true/false
head_movement.enabled	Movement compensation	true for MEG
line_freq	Power line frequency	50 or 60 Hz
interactive_*	User interaction	true for manual, false for batch
autoreject.cv_folds	Cross-validation robustness	3-10 (5 typical)
autoreject.thresh_method	Optimization algorithm	'bayesian_optimization' or 'random_search'
autoreject.n_interpolate	Interpolation levels	[0] for detection only
ica_preprocessing.*.resample_hz	ICA computation speed	100-250 Hz
checkpoint.exit_after_checkpoint	Stage control	true for SLURM

Environment Setup

Required Dependencies

```
# Core dependencies
pip install mne>=1.6.0
pip install mne-bids>=0.14
pip install autoreject>=0.4.3
pip install pyyaml>=6.0
pip install numpy>=1.24.0
pip install scipy>=1.10.0
pip install matplotlib>=3.7.0

# Optional but recommended
```

```
pip install pandas>=2.0.0
pip install joblib>=1.3.0
pip install scikit-learn>=1.3.0
```

Environment Variables

```
# Performance optimization
export OMP_NUM_THREADS=8
export MKL_NUM_THREADS=8
export OPENBLAS_NUM_THREADS=8
export NUMEXPR_NUM_THREADS=8

# Display configuration
export QT_QPA_PLATFORM=xcb # For Linux
export MPLBACKEND=QtAgg    # For interactive
# or
export MPLBACKEND=Agg      # For headless

# MNE configuration
export MNE_USE_CUDA=false  # Unless GPU available
export MNE_LOGGING_LEVEL=INFO
```

File Organization

Input Structure (BIDS)

```
 bids_root/
├── dataset_description.json
├── participants.tsv
├── sub-001/
│   ├── ses-01/
│   │   └── meg/
│   │       ├── sub-001_ses-01_task-experiment_run-01_meg.fif
│   │       ├── sub-001_ses-01_task-experiment_run-01_meg.json
│   │       └── sub-001_ses-01_task-experiment_run-01_channels.tsv
│   └── ses-02/
│       └── meg/
│           └── ...
└── sub-002/
    └── ...
```

Output Structure

```

derivatives/
└─ preprocessing/
    │─ dataset_description.json
    └─ sub-001/
        └─ ses-01/
            └─ meg/
                │─ sub-001_ses-01_task-experiment_run-01_desc-
parproc_meg.fif
                │─ sub-001_ses-01_task-experiment_run-01_desc-
parproc_meg_manifest.yaml
                │─ sub-001_ses-01_task-experiment_run-01_desc-
preproc_meg.fif
                │─ sub-001_ses-01_task-experiment_run-01_desc-
preprocICAeeg.fif
                │─ sub-001_ses-01_task-experiment_run-01_desc-
preprocICAmeg.fif
                │─ sub-001_ses-01_task-experiment_run-01_log.yaml
                │─ sub-001_ses-01_task-experiment_run-01_log.json
                └─ plots/
                    │─ psd_pre_maxwell.png
                    │─ psd_post_maxwell.png
                    │─ head_movement.png
                    │─ ica_components_eeg.png
                    │─ ica_components_meg.png
                    └─ ...

```

Version History

Version	Date	Changes
1.3	2025-09-02	Enhanced AutoReject documentation, improved QC metrics
1.2	2025-08-15	Added hybrid mode, expanded Stage 1/2 separation
1.1	2025-07-01	Checkpoint system implementation
1.0	2025-06-01	Initial release

References and Resources

Core Technologies:

- MNE-Python: <https://mne.tools>
- MNE-BIDS: <https://mne.tools/mne-bids>
- AutoReject: <https://autoreject.github.io>

Methodology Papers:

- Jas et al. (2017). "Autoreject: Automated artifact rejection for MEG and EEG data"
- Taulu & Simola (2006). "Spatiotemporal signal space separation method"
- Hyvärinen & Oja (2000). "Independent component analysis: algorithms and applications"

Community Resources:

- MNE Forum: <https://mne.discourse.group>
 - FieldTrip Comparison: <https://www.fieldtriptoolbox.org>
 - EEGLAB Documentation: <https://eeglab.org>
-

End of Documentation